CSC-239 *Java Programming*

Chapter 12: Exception Handling and Text Input / Output

This lecture is being recorded.

October 22, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Project 1 was due Oct. 15

- Project 1 was due on Wednesday, October 15.
- As stated in the *Syllabus*, the grade penalty for turning in a project late is:

  1% per day (after a 1-day grace period), up to a maximum grade penalty of 20%.

  Any project that is 21 days late (or more) receives a grade of ZERO.
- What this means is: if you need a few extra days to do a good job on your project, then the late penalty will not be very much.
- However, if you need significantly more time than a few days, you should meet with the instructor to get some help.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

- BHCC Tutoring:

  https://bhcc.edu/tutoring/

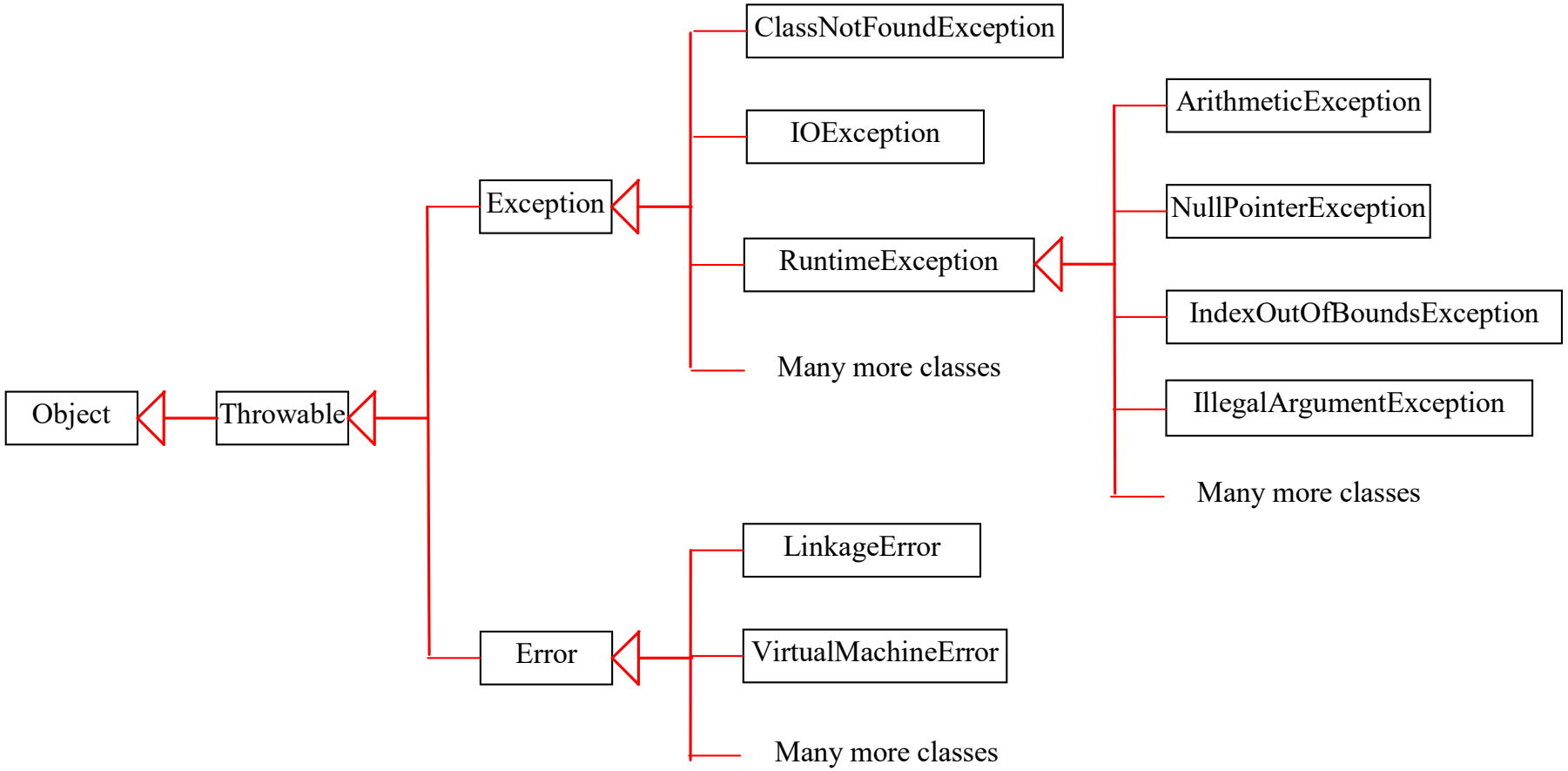
Recall Earlier Lecture

- The compiler can detect syntax errors and some logic errors, but certainly not all runtime errors.
- Some runtime errors are “fatal” errors: the JVM outputs some message and the program stops.
- Exception Handling provides a way to handle some runtime errors *within* the program, so execution can continue.

Exceptions

- Conceptually, an “exception” is an error or other “not normal” occurrence.
- Many possible causes:
  - An exception may be caused by an error in the program.
  - An exception may be caused by some other external occurrence.
  - The root cause may be something as mundane as the user misspelling the name of a file.
- We can write our program to catch and handle many exceptions.
- If our program does not handle the exception, then the program execution halts.

Types of Exceptions



Checked Exceptions vs. Unchecked Exceptions

- For *Checked* Exceptions, the compiler requires either:
  - use a try-catch block, or
  - "declare" the exception.
- *Unchecked* Exceptions:
  - May represent circumstances which the caller cannot be expected to handle, such as an unrecoverable system failure.
  - May represent elementary programming errors, and requiring the code to handle them would make the code too difficult to read.
- See also:
  http://docs.oracle.com/javase/tutorial/essential/exceptions/runtime.html

Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as *declaring exceptions*.

```
public void myMethod()
    throws IOException


public void myMethod()
    throws IOException, OtherException
```

Throwing Exceptions

- When the program detects an error, the program can create an instance of an appropriate exception type and throw it.
- This is known as *throwing an exception*.

Throwing Exceptions Example

- Suppose you have a class that represents circles.
- One operation that may be required is to set the circle's radius value:

```
/** Set a new radius */
  public void setRadius(double newRadius)
      throws IllegalArgumentException {
    if (newRadius >= 0)
      radius =  newRadius;
    else
      throw new IllegalArgumentException(
        "Radius cannot be negative");
  }
```

Catching Exceptions

```
try {
        statements; // (may throw exceptions)
}
catch (Exception1 exVar1) {
        handler for exception1;
}
catch (Exception2 exVar2) {
        handler for exception2;
}
...
catch (ExceptionN exVarN) {
        handler for exceptionN;
}
```

Pros/Cons When Using Exceptions

- Separates error-handling code from normal programming tasks
  - Hopefully makes code easier to read
  - May make the code easier to modify
- There are "overhead" costs associated with exceptions:
  - Instantiating a new exception object
  - Rolling back the call stack
  - Propagating the errors to the calling methods

When to Throw Exceptions

- If an exception (error or unusual event) occurs in a method, and you want the error to be handled by the caller of the method, then the method should throw an exception.
- If you can handle the error in the same method where it occurs, then there is usually no need to throw an exception.

Example: Bad use of Exception (1)

- If a simple, expected condition can be easily detected and handled in the current method, then do NOT use an exception.
  - The following code is not a good use of exception:

```
try {
   System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
   System.out.println("refVar is null");
}
```

Example: Bad use of Exception (2)

- It would be better to replace the code in the previous slide with the following:

```
if (refVar != null)
   System.out.println(refVar.toString());
else
   System.out.println("refVar is null");
```

Defining Custom Exception Classes

- You can define a custom exception class by extending the `java.lang.Exception` class.
- However, you should use the predefined exception classes whenever they are a "good fit" for the error you need to report.
- Define custom exception classes only if the predefined classes are not sufficient.

The `File` class

- A *filename* is a string.
- The File class is a wrapper class for the file **name** and its directory **path**.
- Constructing a File instance does *not* create an actual "file" on the machine.
- You can create a File instance for any file name regardless whether the actual "file" exists or not.
- You can invoke the **`exists()`** method on a File instance to check whether the "file" exists.
- A File object encapsulates the properties of a *filename* or a path, but does *not* contain the methods for reading/writing data from/to an actual "file".

Text Input/Output (I/O)

- In order to perform input or output, you need to create objects using appropriate Java I/O classes.
- PrintWriter class: writes data to a file.
- Scanner class: reads data from a file.
- Sample programs:

  `WriteData.java`

  `ReadData.java`

How Does Scanner Work?

- Token-reading Methods: Read text separated by delimiters.
- Delimiters are white space, by default
- Change the delimiters with

```
useDelimiter(regex)
```

- Example token-reading methods

```
nextByte(), nextShort(), nextInt(),
nextLong(), nextFloat(), nextDouble(),
next()
```

- If the actual input is wrong, this results in an exception:
  `java.util.InputMismatchException`

Reading an Entire Line

- An object of the Scanner class can also read an entire *line* at a time.

```
input = new Scanner(dataFile);
while (input.hasNextLine()) {
      String inputText = input.nextLine();
      lineNum++;
      // (continue processing . . .)
}
```

Examples of arrays:

In previous class sessions we have seen that we can create an array of a primitive data type, such as

```
double[] numbers = new double[100];
```

This is an array of 100 **double** variables, with a default value of `0.0`.

It is also possible to create an array of **String** objects:

```
String[] cities = new String[100];
```

Actually, this is an array of **String** *reference variables*, each having a default value of **null**.

The ArrayList Class

But the **size** of an array cannot be changed after it is created.

Think of an **ArrayList** like an array that has an adjustable size. (See also section 11.11.)

We can use an **ArrayList** with any *reference type*, but not with primitive types.

The ArrayList Class

You can create an array to store objects. But the array's size is fixed once the array is created. Java provides the **ArrayList** class that can be used to store an unlimited number of objects.

java.util.ArrayList<E>	
`+ArrayList()`	Creates an empty list.
`+add(o: E) : void`	Appends a new element o at the end of this list.
`+add(index: int, o: E) : void`	Adds a new element o at the specified index in this list.
`+clear(): void`	Removes all the elements from this list.
`+contains(o: Object): boolean`	Returns true if this list contains the element o.
`+get(index: int) : E`	Returns the element from this list at the specified index.
`+indexOf(o: Object) : int`	Returns the index of the first matching element in this list.
`+isEmpty(): boolean`	Returns true if this list contains no elements.
`+lastIndexOf(o: Object) : int`	Returns the index of the last matching element in this list.
`+remove(o: Object): boolean`	Removes the element o from this list.
`+size(): int`	Returns the number of elements in this list.
`+remove(index: int) : boolean`	Removes the element at the specified index.
`+set(index: int, o: E) : E`	Sets the element at the specified index.

Generic Type

ArrayList is known as a generic class with a generic type E. You can specify a concrete type to replace E when creating an ArrayList. For example, the following statement creates an ArrayList and assigns its reference to variable cities. This ArrayList object can be used to store strings.

```
ArrayList<String> cities = new ArrayList<String>();
```

```
ArrayList<String> cities = new ArrayList<>();
```

Differences and Similarities between Arrays and ArrayList

Operation	*Array*	*ArrayList*
Creating an array/ArrayList	`String[] a = new String[10]`	`ArrayList<String> list = new ArrayList<>();`
Accessing an element	`a[index]`	`list.get(index);`
Updating an element	`a[index] = "London";`	`list.set(index, "London");`
Returning size	`a.length`	`list.size();`
Adding a new element		`list.add("London");`
Inserting a new element		`list.add(index, "London");`
Removing an element		`list.remove(index);`
Removing an element		`list.remove(Object);`
Removing all elements		`list.clear();`

The **`java.util.Arrays`** Class

- The **`java.util.Arrays`** class contains various static methods for manipulating arrays (such as sorting and searching).
  - See also section 7.12.

- The **`Arrays`** class is part of the **`java.util`** package, so your program may need to use an import statement:

```
import java.util.Arrays;
```

Example: **`Arrays.sort()`**

Since sorting is frequently used in programming, Java provides several overloaded sort methods for sorting an array of int, double, char, short, long, and float in the **`java.util.Arrays`** class.

- The following example sorts an array of numbers:

```
double[] numbers={6.0,4.4,1.9,2.9,3.4,3.5};
java.util.Arrays.sort(numbers);
```

- The following example sorts an array of characters:

```
char[] chars = {'a','A','4','F','D','P'};
java.util.Arrays.sort(chars);
```

Create ArrayList from an array

Creating an **ArrayList** from an array of objects:

```
String[] array = {"red","green","blue"};
ArrayList<String> list =
  new ArrayList<>(Arrays.asList(array));
```

Create an array from an ArrayList

Creating an array of objects from an ArrayList:

```
String[] array1 =
        new String[list.size()];
list.toArray(array1);
```